Universidad Católica Andrés Bello
Facultad de Ingeniería
Escuela de Ingeniería Informática
Sistemas operativos

Proyecto Fase 1

Diseño e Implementación de un Simulador de Sistema Operativo: CLI y Gestión de Procesos

Profesor:

Marcel J. Castro G

Alumnos:

Victoria Carmona | C.I.: 30.407.738

Jaiverson Bonillo | C.I.: 31.601.402

Francesco Merola | C.I.: 29.863.369

Caracas, 12 de noviembre de 2026

UCAB Universidad Católica ANDRÉS BELLO

Índice

1. Introducción

2. Estructura modular del código

3. Arquitectura concurrente: hilos y sincronización

4. Estructuras de datos

5. Planificación de procesos y métricas

6. Interfaz de comandos y validación de entradas

7. Pruebas y resultados

8. Compilación y ejecución

9. Gestión del proyecto en GitHub

10. Conclusiones

Referencias bibliográficas

Anexo A. Declaración del uso de Inteligencia Artificial

UCAB Universidad Católica ANDRÉS BELLO

1. Introducción

En este informe presentamos la arquitectura y el desarrollo del simulador de kernel correspondiente a la Fase I del proyecto de Sistemas Operativos. El programa, diseñado como una aplicación de consola en C++17, tiene como objetivo recrear —de manera simplificada pero funcional— dos de las responsabilidades más importantes de cualquier sistema operativo moderno:

- Una interfaz de comandos (shell) interactiva, diseñada para que el usuario pueda crear procesos, cargar lotes de trabajo desde archivos y monitorear el estado del sistema en tiempo real.
- Un motor de planificación (scheduler) encargado de decidir qué proceso ocupa la CPU en cada instante de la simulación, utilizando para ello los algoritmos *Round Robin (RR)* y *Shortest Remaining Time First (SRTF)*.

Dado que ambos componentes operan simultáneamente mediante hilos POSIX independientes y comparten la misma tabla de procesos, nos enfrentamos a dos grandes desafíos técnicos durante el desarrollo. El primero fue garantizar la sincronización: asegurar que los hilos accedieran a la información compartida sin generar bloqueos ni condiciones de carrera. El segundo, igual de crítico, fue lograr una robustez total para blindar el simulador y evitar que colapsara ante errores del usuario, como ingresar letras en lugar de números, dejar comandos por la mitad o intentar leer archivos inexistentes.

Alcance de la entrega: En esta etapa, el proyecto incluye la interfaz de comandos completa, la carga robusta de archivos de traza, el funcionamiento de los algoritmos RR y SRTF, la visualización del diagrama de Gantt en consola y el cálculo automático de métricas. La interfaz gráfica (GUI) propuesta como *bonus* opcional no forma parte del alcance de esta fase.

2. Estructura modular del código

Para garantizar que el proyecto fuera escalable y fácil de mantener por todo el equipo, adoptamos una arquitectura modular basada en la separación de responsabilidades. Cada componente clave del sistema (como la consola, el procesador o la gestión de procesos) está dividido en dos archivos complementarios.

En primer lugar, la carpeta include/ almacena los archivos de cabecera (.h, como pcb.h o cli.h). Estos archivos actúan como "contratos" públicos: exponen únicamente las firmas de las funciones que los demás módulos necesitan conocer para interactuar con ellos, las cuales fueron definidas y acordadas por el equipo al inicio del proyecto. En segundo lugar, la carpeta src/ contiene los archivos fuente (.cpp), donde reside la implementación real y privada de la lógica de cada módulo (como main.cpp o cli.cpp).

Adicionalmente al código fuente, la estructura se apoya en otras carpetas y archivos fundamentales para la gestión del proyecto:

- El archivo Makefile, ubicado en la raíz, que centraliza las reglas para compilar todo el simulador de forma automatizada.
- El archivo README.md, utilizado para documentar las instrucciones de ejecución y las reglas de colaboración del equipo en GitHub.
- El directorio tests/, dedicado exclusivamente a almacenar los archivos de traza (como los *workloads* de prueba) y los escenarios diseñados para evaluar la robustez del sistema frente a errores.
- El directorio docs/, destinado a la documentación formal, incluyendo el presente informe.

Módulo	Responsabilidad
pcb.h / pcb.cpp	Define el PCB, la tabla de procesos, la cola de LISTOS, el diagrama de Gantt y el estado global g_kernel con su mutex.
main.cpp	Inicializa el kernel, crea los dos hilos, espera a que terminen y libera los recursos.
cli.h / cli.cpp	Hilo CLI: lectura, validación y ejecución de comandos; carga de archivos (load); manejo de señales.
utils.h / utils.cpp	Conversión segura de texto a número, separación de una línea en palabras y recorte de espacios.
cpu.h / cpu.cpp	Hilo CPU: reloj simulado, admisión de procesos y algoritmos RR y SRTF.
report.h / report.cpp	Comandos de consulta ps, pcb, gantt, stats y registro del diagrama de Gantt.
Makefile	Compilación normal y versiones de diagnóstico con detectores de errores de memoria y de concurrencia.
tests/	Archivos de traza de ejemplo y archivo de entradas maliciosas (ataques.txt).

Una de las decisiones de diseño más importantes para el éxito de este simulador fue establecer un aislamiento estricto entre los dos hilos principales del programa. Por regla general de nuestra arquitectura: el hilo de la consola (CLI) y el hilo del planificador (CPU) nunca se comunican ni se llaman directamente entre sí.

Toda la interacción entre ambos ocurre a través de un "estado compartido" centralizado en el módulo pcb.h, el cual almacena la tabla de procesos, las colas de listos y las banderas globales del sistema.

Para ilustrar esto, tomemos como ejemplo el comando start. Cuando el usuario lo escribe en la consola, la CLI no "ejecuta" la CPU ni invoca sus funciones. Su única responsabilidad es modificar una bandera en el estado compartido indicando que la simulación ha

comenzado. Por su parte, el hilo de la CPU (que se encuentra monitoreando ese estado) detecta el cambio y arranca el motor de planificación.

Esta decisión de diseño nos aportó dos beneficios fundamentales:

1. Independencia total de los módulos: Al estar desacoplados, los cambios en un módulo no rompen el otro. Por ejemplo, si en el futuro se decide reemplazar la consola de texto por una interfaz gráfica (GUI), el código de la CPU se mantendrá exactamente igual, ya que solo depende del estado en pcb.h.
2. Sincronización segura e infalible: Al concentrar toda la información en un solo lugar, pudimos protegerla fácilmente utilizando un único *Mutex* (KernelLock). De esta forma, centralizamos la sincronización: si la CLI está realizando una carga masiva de procesos (comando load), el candado asegura que la CPU no intente leer a medias esa información, erradicando por completo las temidas condiciones de carrera.

3. Arquitectura concurrente: hilos y sincronización

3.1 Los hilos del programa El núcleo de nuestro simulador se sostiene sobre tres hilos POSIX (pthread) que operan en conjunto. Dos de ellos corresponden a los requisitos centrales del enunciado, mientras que el tercero funciona como el director de la ejecución:

- **Hilo Principal (main):** Es el responsable de preparar el terreno. Crea los mecanismos de sincronización (el mutex y la variable de condición) y lanza los otros dos hilos para que trabajen en paralelo utilizando pthread_create. Una vez que el sistema está corriendo, este hilo se queda esperando pacientemente (con pthread_join) a que los demás terminen su trabajo para, finalmente, liberar la memoria y los recursos del sistema.
- **Hilo de la Consola (CLI):** Es la cara interactiva del simulador. Su tarea es mantener vivo el indicador (>), leer las instrucciones del usuario, validarlas y ejecutar los comandos correspondientes. Este hilo trabaja ininterrumpidamente hasta que el usuario ingresa el comando exit o envía la señal de fin de entrada (Ctrl+D).
- **Hilo del Procesador (CPU):** Es el motor temporal del sistema. Actúa como un reloj: en cada *tick*, revisa si hay procesos nuevos que admitir, ejecuta un paso del algoritmo de planificación que esté activo (RR o SRTF) y avanza el tiempo de la simulación. Este hilo se detiene únicamente cuando recibe la señal de apagado general.

3.2 Exclusión mutua: un solo cerrojo para el estado compartido Toda la información vital que comparten los hilos —como la tabla de procesos, la cola de listos, el tiempo del reloj, el historial del diagrama de Gantt y las banderas de control— reside en una única estructura global llamada g_kernel. Para protegerla de accesos simultáneos, utilizamos un único candado (pthread_mutex_t).

En términos sencillos, este mutex funciona como la llave de una habitación privada: solo el hilo que tiene la llave puede entrar a leer o modificar los datos. Si el otro hilo necesita entrar, debe esperar a que el primero salga y devuelva la llave. Decidimos usar un solo cerrojo global en lugar de varios por dos motivos de diseño: elimina por completo el riesgo de un interbloqueo (*deadlock*) y, dado que las operaciones internas son extremadamente rápidas, el tiempo de espera entre hilos resulta imperceptible.

Para blindar este mecanismo y evitar que un hilo olvide devolver la llave, implementamos la clase KernelLock basada en el patrón RAII de C++. Al crear el objeto, este toma el cerrojo automáticamente, y al salir de su bloque (llaves { }), lo libera por sí solo, incluso si el código fue interrumpido por un return o un error.

3.3 Reglas de concurrencia del equipo Para que los tres integrantes del equipo pudiéramos programar en paralelo sin causar colisiones en la memoria, establecimos cinco reglas de oro documentadas tanto en el repositorio como en el código fuente:

1. Todo acceso a g_kernel (ya sea para leer o escribir) debe hacerse estrictamente dentro de un bloque protegido por KernelLock.
2. Las funciones cuyo nombre termina en _locked asumen que el candado ya está puesto. El hilo que las llama debe tener la llave; si la función intentara pedirla de nuevo, el hilo se bloquearía a sí mismo.
3. Las colas de procesos guardan únicamente números de identificación (PID), no punteros. Un puntero hacia un proceso solo es seguro mientras se mantiene el cerrojo cerrado.
4. Jamás se debe pausar el programa o esperar a que el usuario escriba algo mientras se tiene el candado puesto. Leer archivos o dormir el procesador se hace siempre "afuera" de la sección crítica para no congelar al otro hilo.
5. El hilo CPU tiene prohibido imprimir texto en pantalla. Toda la impresión se delegó al hilo CLI para garantizar que los mensajes no se mezclen visualmente en la terminal.

3.4 Pausa sin espera activa: variable de condición Un desafío importante era manejar la pausa del simulador sin que el hilo CPU desperdiciara recursos de la computadora preguntando constantemente si ya podía arrancar (fenómeno conocido como espera activa). Para solucionarlo de forma elegante, utilizamos una variable de condición (pthread_cond_t).

Cuando la simulación se pausa, el hilo CPU ejecuta pthread_cond_wait, lo cual suelta el candado y pone al hilo a dormir profundamente en un solo movimiento. Cuando el usuario escribe el comando start, la consola cambia la bandera del sistema y ejecuta pthread_cond_signal, "despertando" de inmediato a la CPU. Por seguridad, esta verificación se colocó dentro de un ciclo while (y no un if) para evitar que el hilo reanude el trabajo si ocurre un despertar accidental del sistema operativo.

Además, cuando el hilo CPU necesita pausarse entre un *tick* y otro, divide ese tiempo de espera en fragmentos cortos de 10 milisegundos (siempre sin el candado puesto). Así, si el

usuario decide salir del programa abruptamente, el sistema detecta la orden en menos de 10 milisegundos y cierra de inmediato, sin importar qué tan largos sean los *ticks*.

3.5 Apagado limpio La secuencia de cierre fue diseñada para no dejar rastros en la memoria. Al recibir el comando exit, la consola invoca la función kernel_request_shutdown(), la cual enciende una bandera de salida y transmite un aviso general (pthread_cond_broadcast) para despertar a la CPU en caso de que estuviera dormida.

Al despertar, la CPU detecta la bandera y termina pacíficamente su ciclo. El hilo principal, que estaba aguardando en las sombras, recupera el control de ambos hilos y, solo cuando está 100% seguro de que ya nadie los está utilizando, destruye el mutex y la variable de condición, garantizando un apagado perfecto.

3.6 Implementación del planificador y del hilo CPU

1. **Hilo CPU y reloj simulado**
   El sistema utiliza un hilo independiente que ejecuta ciclos de CPU. En cada tick:

- Admite procesos cuyo tiempo de llegada ya se alcanzó;
- Selecciona el siguiente proceso;
- Ejecuta una unidad de ráfaga;
- Actualiza tiempos y el diagrama de Gantt;
- Incrementa el reloj simulado.

2. **Algoritmos de planificación**

- Round Robin (RR): selecciona el proceso al frente de la cola y lo interrumpe cuando consume su quantum.
- SRTF: selecciona el proceso con menor tiempo de ráfaga restante y permite la apropiación si llega otro proceso más corto.
- En caso de empate, SRTF utiliza como criterios el tiempo de llegada y posteriormente el PID.

3. **Estados de los procesos**

   Explicar las transiciones NEW → READY → RUNNING → TERMINATED, incluyendo que los procesos futuros permanecen en NEW hasta que llega su tiempo de entrada.

4. **Métricas y estadísticas**

   cpu.cpp actualiza:

- Tiempo de espera (waiting_time);
- Tiempo de primera ejecución (first_run_time);
- Tiempo de finalización (finish_time);

- Ticks ocupados de CPU (busy_ticks);
- Segmentos del diagrama de Gantt.

5. **Sincronización entre hilos**
   Se puede documentar el uso de KernelLock, pthread_cond_wait y pthread_cond_broadcast. El hilo CPU permanece dormido cuando el sistema está pausado y se despierta al ejecutar start o exit.

6. **Respuesta ante la terminación**
   La función cpu_sleep_interruptible() divide la espera en intervalos de 10 ms para detectar rápidamente una solicitud de salida.

4. Estructuras de datos

4.1 Bloque de Control de Proceso (PCB)

Cada proceso se representa con una estructura PCB que guarda su identidad, su estado y los tiempos necesarios para calcular las métricas. Todos los tiempos se miden en ticks del reloj simulado.

Campo	Significado
pid	Identificador único, asignado de forma incremental desde 1.
name	Nombre del proceso (1 a 32 caracteres: letras, dígitos, _, - o .).
state	Estado actual: NEW, READY, RUNNING, BLOCKED o TERMINATED.
priority	Prioridad de 1 a 5 (por defecto 3).
arrival_time	Tick en el que el proceso llega al sistema.
burst_initial / burst_remaining	Ráfaga total de CPU solicitada y la que todavía le falta.
first_run_time	Primer tick en que usó la CPU (–1 si nunca la ha usado). Se usa para el tiempo de respuesta.
finish_time	Tick en que terminó (–1 si no ha terminado). Se usa para el tiempo de retorno.
waiting_time	Ticks acumulados esperando en la cola de LISTOS.
quantum_used	Ticks consumidos del quantum actual (solo Round Robin).

4.2 Tabla de procesos

Los PCB se guardan en un std::vector<PCB> donde la posición de cada proceso es pid – 1, lo que permite encontrar cualquier proceso de forma inmediata. Al iniciar el programa se reserva espacio para los 1000 procesos permitidos. Esto evita un error clásico: si el vector

creciera, C++ movería todos los datos a otra zona de memoria y cualquier puntero que apuntara a un PCB quedaría inválido, provocando un *segmentation fault*.

4.3 Cola de LISTOS

La cola de procesos listos es un std::deque<int> que almacena PIDs, no punteros ni copias de los PCB. Así cada proceso existe en un solo lugar (la tabla) y la cola solo indica el orden. Se eligió deque porque permite sacar del frente y agregar al final de forma eficiente, que es exactamente lo que necesita Round Robin, y también permite recorrerla completa para buscar el proceso de menor ráfaga en SRTF.

4.4 Diagrama de Gantt

El historial de ejecución se guarda como una lista de tramos {inicio, fin, pid}. En cada tick, el hilo CPU registra qué proceso se ejecutó (o la CPU ociosa, *IDLE*). Si es el mismo proceso que en el tick anterior, en lugar de crear un tramo nuevo se extiende el último. Así el comando gantt muestra directamente tramos como [1-3s: Job_B].

4.5 Ciclo de vida de un proceso

El estado BLOCKED está definido en la estructura, pero no se utiliza en esta fase, ya que la simulación no incluye operaciones de entrada/salida. Queda preparado para fases posteriores.

5. Planificación de procesos y métricas

5.1 El reloj simulado

La simulación avanza en unidades llamadas ticks. La duración real de un tick se ajusta con set_speed (de 50 a 5000 ms), pero los cálculos siempre se hacen en ticks. Si el reloj vale t, en cada iteración el hilo CPU realiza, en este orden y con el mutex tomado:

1. Admisión: los procesos en estado NEW cuyo arrival_time ≤ t pasan a READY y entran a la cola.
2. Ejecución: el algoritmo activo elige el proceso y simula el intervalo [t, t+1).
3. Avance: el reloj pasa a t+1.

Por esta convención, un proceso que consume su último tick en el intervalo [t, t+1) tiene finish_time = t+1.

5.2 Round Robin (RR)

Cada proceso usa la CPU como máximo durante un **quantum** (2 ticks por defecto, configurable con set_sched RR <quantum>). Si al agotar su quantum todavía le queda ráfaga, el proceso vuelve **al final** de la cola de LISTOS y pasa el siguiente. Es un algoritmo justo: ningún proceso acapara la CPU.

Si un proceso llega y en ese mismo tick otro proceso agota su quantum, la nueva llegada queda antes que el proceso desalojado. Ya que primero se admiten las nuevas llegadas en (arrival_time <= clock_ticks) y se agregan a la cola por orden de llegada y luego por PID y después el proceso que se estaba ejecutando, se cambia su estado a READY y se agrega al final de la cola.

5.3 Shortest Remaining Time First (SRTF)

En cada tick se ejecuta el proceso al que le queda menos ráfaga. Es apropiativo: si llega un proceso con menor ráfaga restante que el que está ejecutando, este es desalojado de inmediato y vuelve a la cola. En caso de empate se elige el que llegó primero y, si también coinciden, el de menor PID. Minimiza el tiempo de espera promedio, aunque los procesos largos pueden esperar mucho si siguen llegando procesos cortos.

El Escenario 2 del enunciado ilustra la desapropiación:

Intervalo	Llegadas	Ráfaga restante en ese momento	Se ejecuta
[0-1)	Job_A (4)	A = 4	Job_A
[1-3)	Job_B (2)	A = 3, B = 2	Job_B (desaloja a A)
[3-4)	Job_C (1)	A = 3, C = 1	Job_C
[4-7)	—	A = 3	Job_A

5.4 Cambio de algoritmo en caliente

El planificador puede cambiarse sin reiniciar: pause detiene el reloj, set_sched cambia el algoritmo dentro de una sección crítica y start reanuda. Los procesos conservan su ráfaga restante y su posición en la cola.

5.5 Métricas

Al finalizar los procesos, el comando stats calcula los promedios de las siguientes métricas:

Métrica	Fórmula	Qué mide
Tiempo de retorno (TAT)	finish_time – arrival_time	Cuánto tardó el proceso desde que llegó hasta que terminó.

Tiempo de espera (WT)	Ticks acumulados en READY (= TAT – burst_initial)	Cuánto tiempo estuvo listo pero sin usar la CPU.
Tiempo de respuesta (RT)	first_run_time – arrival_time	Cuánto tardó en recibir la CPU por primera vez.
Uso de CPU	busy_ticks / ticks totales × 100	Porcentaje del tiempo en que la CPU no estuvo ociosa.

Como referencia para validar la implementación, estos son los valores esperados del Escenario 2, calculados a mano:

Proceso	Llegada	Ráfaga	Fin	TAT	WT	RT
Job_A	0	4	7	7	3	0
Job_B	1	2	3	2	0	0
Job_C	3	1	4	1	0	0
Promedio				**3,33**	**1,00**	**0,00**

La CPU estuvo ocupada los 7 ticks, por lo que el uso de CPU esperado es del 100 %.

6. Interfaz de comandos y validación de entradas

Como el enunciado indica que la evaluación incluye intentos deliberados de romper el programa, la CLI se diseñó bajo un principio simple: ninguna entrada del usuario puede terminar el programa ni dejar el mutex tomado. Ante cualquier error se muestra un mensaje explicativo y se vuelve a pedir el siguiente comando.

6.1 Recorrido de un comando

1. Se lee la línea completa con getline (sin cerrojo). Si supera 512 caracteres, se rechaza.
2. Se separa en palabras con tokenize, que ignora espacios y tabulaciones repetidos.
3. Se busca el comando en una tabla de despacho que asocia cada nombre con su función. Si no existe, se informa; si existe pero está escrito en mayúsculas (CREATE), se sugiere la forma correcta.
4. La función del comando valida cantidad de argumentos (check_argc), luego tipo y rango de cada número (get_int_arg), y solo entonces toma el cerrojo para modificar el estado.
5. El resultado se imprime después de soltar el cerrojo.

6.2 Conversión segura de texto a número

Las funciones habituales de C/C++ para convertir texto en número no son seguras para este propósito: atoi("abc") devuelve 0 sin avisar, atoi("3.5") devuelve 3, y stoi("abc") lanza una excepción que, si no se captura, termina el programa. Por eso el equipo prohibió su uso y creó parse_int, que trabaja en dos fases: primero comprueba carácter por carácter

que el texto tenga la forma de un entero (signo opcional y solo dígitos), y después lo convierte con strtoll, detectando desbordamientos.

Entrada	atoi()	parse_int()
5	5	OK → 5
abc	0 (sin aviso)	NO ES UN ENTERO
3.5	3 (sin aviso)	NO ES UN ENTERO
5abc	5 (sin aviso)	NO ES UN ENTERO
1e3	1 (sin aviso)	NO ES UN ENTERO
99999999999999999999	valor indefinido	FUERA DE RANGO

6.3 Tipos de error

Mensaje	Cuándo aparece	Ejemplo
[ERROR DE SINTAXIS]	Faltan o sobran argumentos. Muestra el uso correcto.	create P1
[ERROR DE TIPO]	Se esperaba un número entero y se recibió otra cosa.	create P1 abc
[ERROR DE PARAMETRO]	El valor es del tipo correcto pero está fuera de rango o no existe.	set_sched FIFO, pcb 99
[ERROR DE FORMATO]	Una línea de un archivo de traza es inválida (indica el número de línea).	Job_X, 4
[ERROR]	Comando desconocido, archivo inexistente, carpeta en lugar de archivo, etc.	load include

Un caso que muestra el nivel de detalle: pcb 1001 no es un PID inválido, sino un PID que no existe. Por eso se reporta como error de parámetro, mientras que pcb abc, pcb 0 o pcb -1 se reportan como error de tipo, tal como pide el enunciado al exigir que el PID sea un entero positivo y que exista.

6.4 Carga por lotes (comando load)

El comando load lee un archivo donde cada línea describe un proceso con el formato NOMBRE, RÁFAGA, PRIORIDAD, LLEGADA. Su funcionamiento se divide en cuatro etapas:

1. Verificar la ruta con la llamada al sistema stat(): se rechazan rutas inexistentes, carpetas (S_ISDIR) y archivos especiales como /dev/null (!S_ISREG). Sin esta comprobación, Linux permite abrir una carpeta y el programa informaba "0 procesos cargados" sin explicar el problema.
2. Leer y validar todo el archivo sin tomar el cerrojo. Las líneas vacías y las que empiezan con # se ignoran. Cada línea se separa por comas con una regla estricta: n comas producen siempre n+1 campos, de modo que una coma sobrante al final se detecta como error. Las líneas inválidas se reportan con su

número y se omiten, sin cancelar el resto; si hay más de 10 errores (por ejemplo, al cargar un archivo binario) se muestra un resumen para no inundar la pantalla.

3. Insertar todos los procesos válidos de una sola vez, dentro de una única sección crítica. Antes se comprueba que haya espacio para todos; si no lo hay, no se carga ninguno. Esta carga atómica garantiza que el hilo CPU nunca vea un archivo cargado a medias.
4. Imprimir el resultado fuera del cerrojo: el PID asignado a cada proceso y el número de líneas omitidas.

Los archivos creados en Windows terminan cada línea con \r\n; el carácter \r sobrante se elimina al recortar cada línea, por lo que estos archivos se cargan correctamente.

6.5 Señales del sistema operativo

Al pulsar Ctrl+C, el sistema operativo envía la señal SIGINT, cuya acción por defecto es terminar el proceso de inmediato. Para evitar un cierre abrupto se instaló un manejador con sigaction:

- El manejador solo muestra un aviso indicando que se use exit. Utiliza write() y no cout, porque un manejador puede ejecutarse en medio de otra escritura y solo las funciones *async-signal-safe* como write() son seguras en ese contexto.
- Se usa la bandera SA_RESTART para que la lectura del teclado (getline) se reanude automáticamente tras la señal en lugar de fallar, lo que el programa habría interpretado como fin de entrada.
- Se trata igual SIGQUIT (Ctrl+\) y se ignora SIGPIPE, que terminaría el programa si su salida se redirige a otro comando que se cierra.

8. Compilación y ejecución

Requisitos: un entorno POSIX (Linux o WSL en Windows) con g++ compatible con C++17 y make. En Ubuntu se instalan con sudo apt install build-essential. El Makefile compila con -pthread y con todas las advertencias activadas (-Wall -Wextra -Wpedantic).

Comando	Para qué sirve
make	Compila el programa y genera el ejecutable simulador.
./simulador	Ejecuta el simulador. Escriba help para ver el manual de comandos.
make run	Compila y ejecuta en un solo paso.
make debug	Compila con AddressSanitizer y UBSan para detectar errores de memoria.
make tsan	Compila con ThreadSanitizer para detectar condiciones de carrera.
make clean	Elimina los archivos generados por la compilación.

9. Conclusiones

El simulador cumple su objetivo central: dos hilos que trabajan simultáneamente sobre información compartida sin condiciones de carrera, comprobado con ThreadSanitizer. La clave fue concentrar todo el estado compartido en una sola estructura protegida por un único mutex, envolver el cerrojo en una clase RAII para que nunca quede tomado, y fijar desde el inicio reglas de concurrencia comunes para todo el equipo.

En cuanto a la robustez, la combinación de una conversión numérica estricta, la validación en capas (cantidad, tipo y rango) y el manejo de señales permitió que el programa resista las 77 entradas maliciosas de prueba sin colapsar.

En cuanto a la planificación, ambos algoritmos se comportaron según lo esperado. Round Robin repartió la CPU por turnos de igual duración, sin que ningún proceso la acaparara, mientras que SRTF priorizó siempre al proceso con menor ráfaga restante y desalojó de inmediato al que estaba en ejecución cuando llegó uno más corto. En el Escenario 2, el diagrama de Gantt coincidió exactamente con la secuencia exigida por el enunciado ([0-1s: Job_A] -> [1-3s: Job_B] -> [3-4s: Job_C] -> [4-7s: Job_A]), y las métricas obtenidas coincidieron con el cálculo manual. Además, se comprobó que el planificador puede cambiarse durante la ejecución sin perder procesos, reiniciando el turno del proceso activo para que la nueva política se aplique de forma correcta desde el primer tick.

UCAB Universidad Católica ANDRÉS BELLO

Referencias bibliográficas

- Arpaci-Dusseau, R. H., y Arpaci-Dusseau, A. C. (2018). *Operating Systems: Three Easy Pieces* (versión 1.00). Arpaci-Dusseau Books. https://pages.cs.wisc.edu/~remzi/OSTEP/
- cppreference.com. (s. f.). *C++ reference: std::deque, std::getline, std::strtoll*. https://en.cppreference.com/
- Kerrisk, M. (2010). *The Linux Programming Interface*. No Starch Press.
- Silberschatz, A., Galvin, P. B., y Gagne, G. (2018). *Operating System Concepts* (10.ª ed.). Wiley.
- Stallings, W. (2018). *Operating Systems: Internals and Design Principles* (9.ª ed.). Pearson.
- Tanenbaum, A. S., y Bos, H. (2015). *Modern Operating Systems* (4.ª ed.). Pearson.
- The Clang Team. (s. f.). *ThreadSanitizer*. https://clang.llvm.org/docs/ThreadSanitizer.html
- The IEEE y The Open Group. (2018). *The Open Group Base Specifications Issue 7 (IEEE Std 1003.1-2017): pthread_mutex_lock, pthread_cond_wait, sigaction*. https://pubs.opengroup.org/onlinepubs/9699919799/

UCAB Universidad Católica ANDRÉS BELLO

Anexo A. Declaración del uso de Inteligencia Artificial

Herramienta: Claude , utilizada por Victoria Carmona.

Forma de uso: la integrante 1(Victoria Carmona) definió los requisitos del proyecto: los archivos y módulos necesarios, el número de hilos, la protección del PCB mediante mutex y la resistencia ante entradas inválidas. A partir de esas instrucciones, la herramienta generó la estructura inicial del código. En el módulo CLI actuó como tutor: guió la implementación por etapas, explicando el porqué de cada decisión, mientras la integrante escribía, compilaba y probaba el código.

Prompts principales (extractos):

- Actúa como un Ingeniero de Software Senior en C++17 y tutor de Sistemas Operativos. Estamos desarrollando un simulador de kernel basado en una arquitectura concurrente con hilos POSIX (CLI y CPU) y un estado compartido protegido por un único Mutex (KernelLock). Te proporcionaré el esqueleto base de nuestros archivos (.h y .cpp). Necesito que me guíes paso a paso para implementar la lógica y la validación estricta de los comandos (como load y pcb), asegurando que el sistema resista cualquier intento de ruptura mediante archivos malformados o datos inválidos. No me generes la solución de golpe; explícame el razonamiento detrás de cada bloque para comprender la concurrencia.

Validación. Ningún resultado se aceptó sin verificarlo: compilación sin advertencias (-Wall -Wextra -Wpedantic), 77 entradas maliciosas sin fallos (tests/ataques.txt), 0 advertencias de ThreadSanitizer, 0 errores de AddressSanitizer y comparación de todas las salidas con el enunciado.